

AIML320

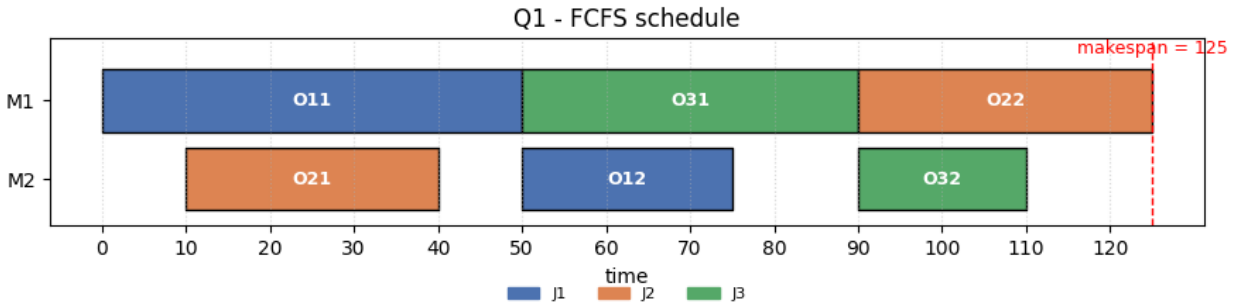
Assignment 3

Andrew Tempany

Victoria University

Part 1: Job Shop Scheduling

Question 1



Operation	Machine	Op ready time	Machine Idle Time	Latest Starting Constraint	Start Time
O ₁₁	M ₁	0	0	Equal	0
O ₂₁	M ₂	10	0	Operation	10
O ₃₁	M ₁	20	50	Machine	50
O ₁₂	M ₂	50	40	Operation	50
O ₂₂	M ₁	40	90	Machine	90
O ₃₂	M ₂	90	75	Operation	90

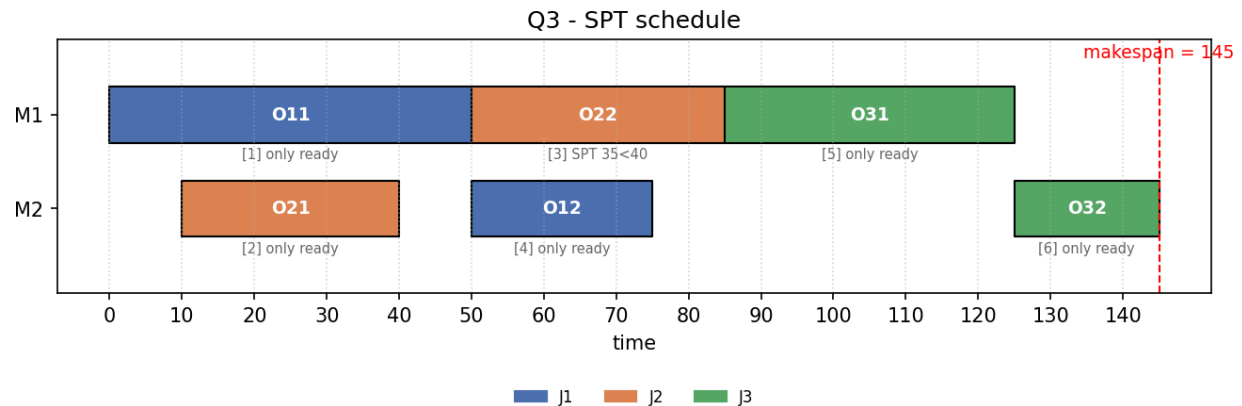
The table above shows the starting times for each operation. The highest value between each operation's "Operation Ready Time" and "Machine Idle Time" has been highlighted to indicate which constraint becomes the starting time.

Question 2

Job	Final Task	Completion Time
J ₁	O ₁₂	75
J ₂	O ₂₂	125
J ₃	O ₃₂	110

The final task to complete is J_2 at time 125. Despite J_2 commencing before J_3 , machine availability becomes the bottleneck. As O_{31} becomes available before O_{22} , M_1 completes O_{22} as its final task.

Question 3



The Gantt chart above shows the processing times for each task following SPT. There is 1 significant change in the order of operations which is M_1 commencing O_{22} before O_{32} . The decisions can be seen in the table below.

Step	Time	Machine Free	Ready & Competing Operations	Decision	Runs
1	0	M_1	O_{11} (proc=50)	O_{11} (only ready op)	0–50
2	10	M_2	O_{21} (proc=30)	O_{21} (only ready op)	10–40
3	50	M_1	O_{22} (proc=35) O_{31} (proc=40)	O_{22} (SPT: 35 < 40)	50–85
4	50	M_2	O_{12} (proc=25)	O_{12} (only ready op)	50–75
5	85	M_1	O_{31} (proc=40)	O_{31} (only ready op)	85–125
6	125	M_2	O_{32} (proc=20)	O_{32} (only ready op)	125–145

This leaves us with the following order of operations:

$Process(O_{11}, M_1, 0) \rightarrow Process(O_{21}, M_2, 10) \rightarrow Process(O_{22}, M_1, 50) \rightarrow Process(O_{12}, M_2, 50) \rightarrow$
 $Process(O_{31}, M_1, 85) \rightarrow Process(O_{32}, M_2, 125)$

Question 4

The makespan of the SPT schedule ends up taking longer, ending at 145 opposed to FCFS' 125.

This is largely due to SPT having an awkward sequence at the end where the constraint of O_{32} being unable to commence until O_{31} is complete. As a result the 2 machines are unable to run simultaneously and the finishing time is delayed. Each job's completion time is displayed in the table below.

Job	1st Operation Completion	2nd Operation Completion
J_1	50	75
J_2	40	85
J_3	125	145

Question 5

In this particular case, FCFS completed first. That doesn't mean the FCFS is ultimately better than SPT as heuristics should be evaluated across many instances, not a single example. In situations with more jobs and more machines, SPT is more likely to compound its gains in productivity and complete more operations in a given timespan than FCFS. However, it will more likely end up with a slower tail, leaving longer operations for last. FCFS is more likely a better rule for situations in which jobs need to be completed sequentially, for example a print shop may want to deliver its prints to clients as soon as they are completed.

Ultimately the better heuristic comes down to the purpose and objectives of the scheduler. FCFS may be a better solution in situations where you want to prioritise the tasks in an order that promotes fairness and earliest due dates per task if all jobs are given the same estimated time to complete. SPT is likely a better choice when you are trying to reduce average wait time across all tasks in any given day.

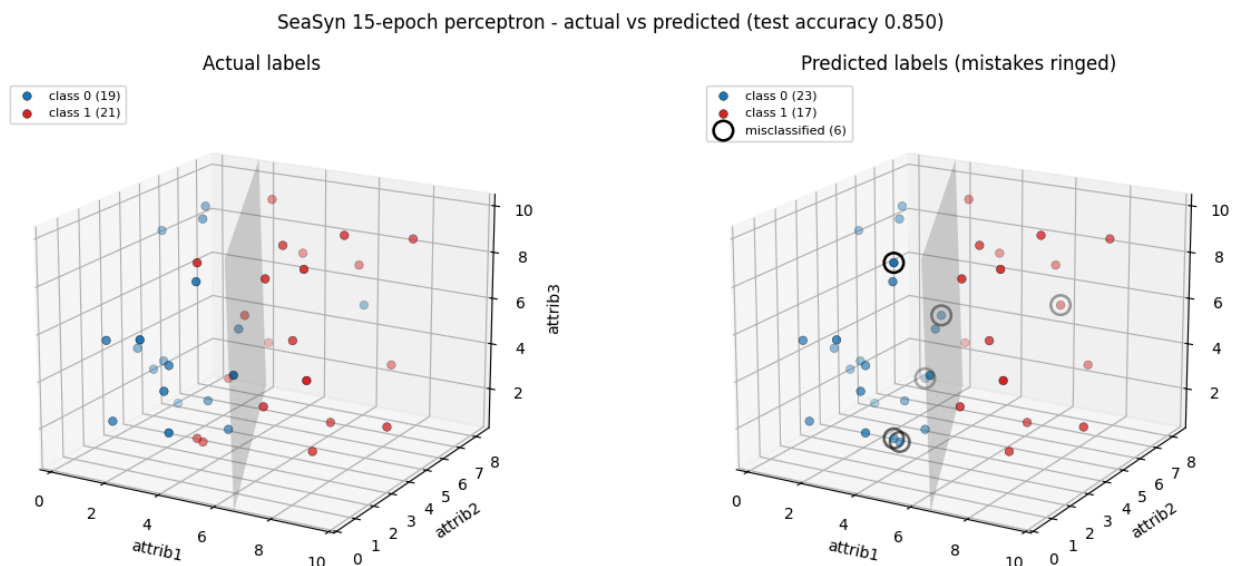
Part 2

Task A: SeaSyn Data Using a Perceptron Model

Number of Epochs	Train Accuracy	Test Accuracy
1	0.65	0.625
5	0.75	0.700
10	0.69	0.600
15	0.85	0.850
20	0.72	0.625
50	0.73	0.650
60	0.71	0.600
80	0.70	0.625
100	0.72	0.625
120	0.75	0.675
150	0.79	0.750
200	0.81	0.750

Our accuracy fluctuates within a range of 0.6 to 0.85 accuracy and generally sits in the 0.625 to 0.75 range. Our 15 epoch perceptron was the best performer, and we don't see a trend of better performance with more epochs.

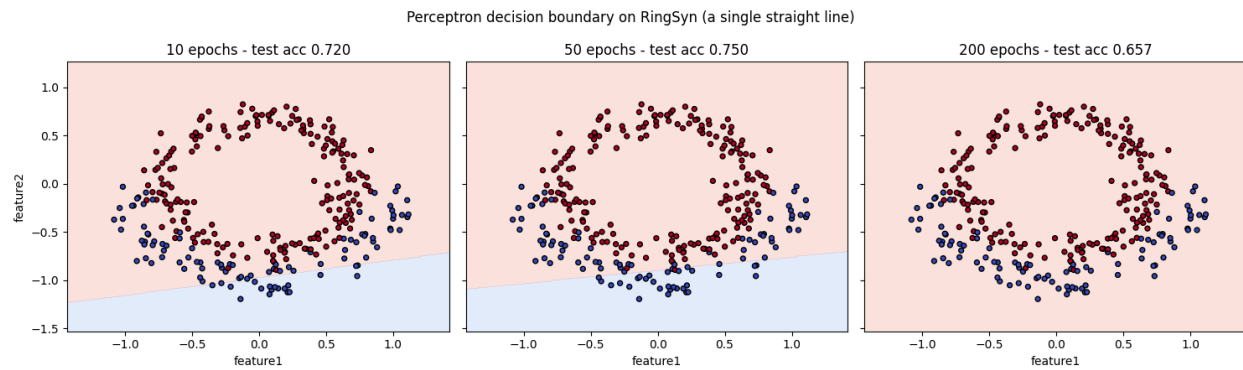
The data doesn't have a perfect linear separation, which is what the perceptron aims to achieve. This means that as it continues to determine weightings between each epoch, instead of converging on a "correct" set of weights, between each epoch it oscillates according to which points were miscategorised in any given epoch. The 15 Epoch perceptron likely landed on a higher accuracy due to chance as opposed to overfitting. This is corroborated by the training data accuracy as well, which also doesn't achieve any higher than 0.85.



Task B: RingSyn Data Using a Perceptron Model

Number of Epochs	Train Accuracy	Test Accuracy
1	0.6689	0.6567
5	0.6689	0.6567
10	0.7241	0.7200
15	0.6689	0.6567
20	0.6689	0.6567
50	0.7506	0.7500
60	0.6689	0.6567
80	0.6689	0.6567
100	0.6689	0.6567
120	0.6689	0.6567
150	0.6689	0.6567
200	0.6689	0.6567

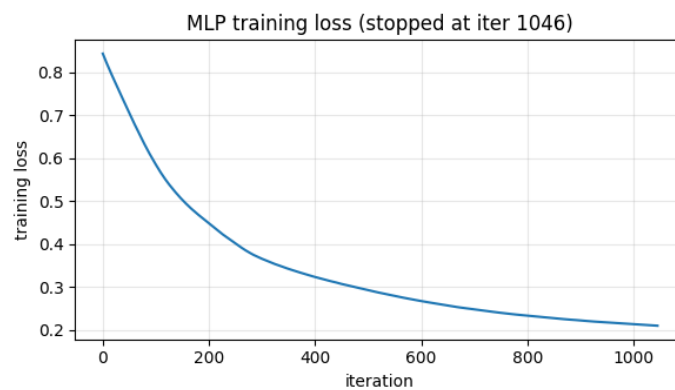
Our RingSyn data was even less amenable to a linear division of labels as it portrays ring shaped data. From the rendering of our 200 epoch test below, we can see that 0.6567 accuracy was achieved when every instance was assigned a single label, we can consider this our baseline accuracy, it was the outcome for 12 of 15 different tests.



Task C: Training an MLP on Non-Linearly Separable Data

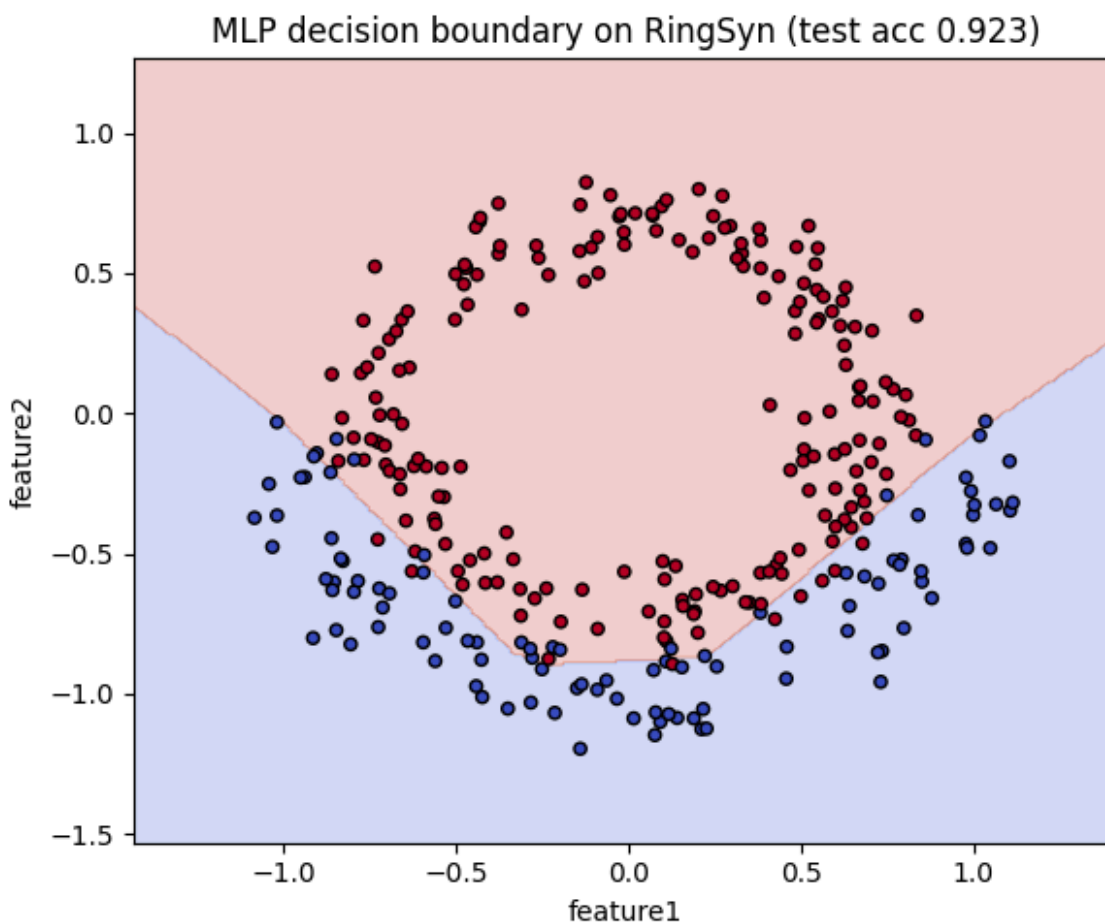
Train Accuracy	0.9227
Test Accuracy	0.9233
Perceptron Test Accuracy	0.7500
Baseline Accuracy	0.6567

Using an MLP marked a significant improvement over the single perceptron and the baseline. We used 1 layer with 8 neurons, a maximum of 2000 iterations and the Relu activation function. Of the maximum iterations the SciKit library hit its default minimum loss threshold on the 1046th epoch.



Our MLP test accuracy reached 0.9233 compared to the single perceptron reaching 0.75 accuracy. So we made an improvement of 0.17 on a single attempt at an MLP.

In the graph rendered below, we can see that the decision boundary has reached an approximate curve around the ring and as a result captures the structure of the data much better and provides more accurate predictions of the cluster labels. Using an MLP allows us to achieve something close to a curved boundary by having each perceptron determine its boundaries with a nonlinear activation function (such as Relu as we've used here) and then being composed by the final output neuron. The single perceptrons will always be limited to a single linear boundary which explains the significant improvement in accuracy.



References and AI Usage

AI was used to assist in writing code for specific tasks. The major use case was in plotting graphs that I ask for and as a syntax/python resource for things like “How do I implement an MLP using the relu activation function using SciKitLearn?”

AI was not used to write this report, or draw conclusions, though it was asked for feedback on my clarity of writing.